

Enumeracion para privesc en Windows

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Enumerar para privesc en Windows es recorrer sus vectores típicos en orden: privilegios de token, grupos, servicios mal configurados, tareas programadas, credenciales guardadas y parches faltantes. WinPEAS lo automatiza; entender cada hallazgo es lo que lo convierte en escalada. Paso previo del winprivesc-modelo.

Windows esconde muchos vectores en configuración (servicios, registro, permisos NTFS) que a mano son tediosos. WinPEAS y las consultas manuales garantizan cobertura, pero dan falsos positivos: hay que validar cada pista.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Nada más conseguir una shell en Windows. Corre primero whoami /priv y whoami /groups, luego WinPEAS para lo exhaustivo.

Cómo identificarla

- whoami /priv revela un privilegio potente (SeImpersonate, SeBackup, SeDebug).
- WinPEAS resalta servicios modificables o rutas sin comillas.
- Credenciales en ficheros de despliegue o en el gestor de credenciales.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

whoami /priv y whoami /groups primero; después winPEAS para el resto.

Evidencia esperada: Un privilegio de token Enabled, o un hallazgo resaltado en winPEAS (servicio, credencial, AlwaysInstallElevated).

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

`whoami /priv`

Objetivo y contexto: Lista tus privilegios de token, el vector mas rapido en Windows. Varios (Selpersonate, SeBackup, SeRestore, SeDebug) llevan a SYSTEM casi directo.

Resultado esperado: Privilegios y si estan Enabled. Selpersonate -> token-impersonation. SeBackup/SeRestore -> sebackup-serestore. Anota los que esten habilitados.

Capa 2: prueba documentada

`systeminfo`

Objetivo y contexto: Muestra version de Windows y hotfixes instalados. Comparar los parches con los CVE conocidos revela si hay un exploit de kernel aplicable.

Resultado esperado: OS version, build y lista de KB (parches). Pocos KB en un Windows viejo -> candidato a exploit de kernel; pasalo por Watson/wesng.

Anatomía de los payloads

La primera prueba es `whoami /priv`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un privilegio de token Enabled, o un hallazgo resaltado en winPEAS (servicio, credencial, AlwaysInstallElevated).

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Checks rapidos: `whoami /priv`, `whoami /groups`, `systeminfo` (parches).
- Lanza WinPEAS y lee lo resaltado (servicios, credenciales, AlwaysInstallElevated).
- Prioriza por rentabilidad: token (token-impersonation) y servicios (service-misconfig-windows) primero.
- Deja los exploits de kernel (kernel-exploits-windows) para el final.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- Ya recorriste privilegios de token, servicios y credenciales sin hallazgos.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.

- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Igual que en Linux pero con otros checks: privilegios de token, servicios, credenciales guardadas y parches faltantes. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (enum-privesc-windows)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.